분야 AI/SW 기초 구분 자료구조와 알고리즘 학습시간 80시간 필수/선택 필수

파일이 언제 어떻게 바뀌었는지 기록하는 작은 프로그램 만들기

문제기술

기술적 설명



1. 미션 소개

Git의 커밋 하나에는 그래프 자료구조와 해시가 담겨 있습니다. 이걸 알고 쓰면 rebase, merge, cherry-pick이 다르게 보이고, 알고리즘 공부와도 연결됩니다. Mini Git을 직접 구현하면서 그 안의 구조를 손으로 확인합니다.

Git은 전 세계 개발자들이 사용하는 분산 버전 관리 시스템입니다.

브랜치로 병렬 작업을 지원하고, 변경 이력을 추적하며, 그 핵심에는 그래프 자료구조와 탐색 알고리즘이 있습니다.

이번 미션에서는 Git의 핵심 구조를 직접 구현하며 CLI 기반 Mini Git을 완성합니다. 커밋 구조를 구축하고 브랜치를 관리하는 기능, 커밋을 검색하고 정렬하는 시스템 등을 만들며 실제 Git의 동작 원리를 체득합니다.

이 경험은 다양한 알고리즘 학습의 토대가 되며 코딩 테스트를 준비하는 데 도움이 됩니다.

2. 최종 결과물

다음 기능이 정상 동작하는 CLI 기반 Mini Git 프로그램 1개를 완성한다.

1. 저장소 및 브랜치 관리

- 입력/요청: INIT <user_name> , BRANCH <branch_name> , SWITCH <branch_name> , COMMIT <message>

- 출력/화면: 저장소 초기화 결과, 브랜치 생성/전환 결과, 커밋 생성 결과(커밋 hash 포함)

2. 커밋 로그 및 탐색

- 입력/요청: `LOG`, `PATH <commit1> <commit2>`, `ANCESTORS <commit_hash>`
- 출력/화면: 커밋 목록 출력, 최단 경로 출력(없으면 `No path`), 조상 목록 출력

3. 검색 및 정렬

- 입력/요청: `SEARCH <keyword>`, `SEARCH --author=<name>`, `LOG --sort-by=date|author`
- 출력/화면: 검색 결과 커밋 목록, 정렬 기준에 따른 로그 출력

4. CLI 인터페이스(REPL)

- 입력/요청: `mini-git>` 프롬프트에서 명령을 반복 입력
- 출력/화면: 명령 파싱 → 실행 → 결과 출력이 반복, `exit` / `quit` 로 종료

5. 필수 제출물 / 실행

- 필수 제출물: 엔트리 포인트(예: `main.py`) 1개 + `README.md` 1개
- 실행(예): `python main.py`

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- 커밋 그래프를 구현하고, Git의 커밋 구조가 왜 DAG인지 말로 설명할 수 있다.
- “부모가 먼저 출력되는 로그”를 만들기 위해 어떤 접근(예: 위상 정렬 성격의 출력)이 필요한지 설명할 수 있다.
- 두 커밋 사이의 최단 경로를 찾는 방법과 특정 커밋의 모든 조상을 탐색하는 방법을 설명할 수 있다.
- 정렬 알고리즘을 직접 구현하고, 평균/최악 시간복잡도 및 안정 정렬 여부를 설명할 수 있다.
- 역색인의 동작 원리와, 순회 검색보다 빠른 이유를 시간복잡도 관점에서 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. CLI 공통 규칙(문법 표준)

- 명령어는 대소문자를 구분하지 않는다. (예: `INIT`, `init` 모두 허용)
- 문자열 인자(사용자명/커밋 메시지/검색 키워드)는 공백을 포함할 수 있다.
    - 공백 포함 시 따옴표로 감싼다. (예: `COMMIT "Add login feature"`)
- 옵션 표기는 아래 형식으로 통일한다.
    - `SEARCH --author=<name>`
    - `LOG --sort-by=date|author`
- 잘못된 입력에 대한 최소 에러 메시지를 표준화한다.
    - 예: `Invalid args`, `Unknown branch: <name>`, `Unknown commit: <hash>`

2. 커밋 그래프(핵심 자료구조)

- 커밋 노드는 최소 필드를 가진다: `hash`, `message`, `author`, `timestamp`, `parents`
- 각 커밋은 0개 이상의 부모 커밋을 가질 수 있다.
- 커밋 그래프는 DAG(방향성 비순환) 구조여야 한다.
- 커밋 저장소는 커밋 hash로 빠르게 찾을 수 있어야 한다(예: 해시맵 기반).
- 커밋 hash는 세션 내 유일해야 한다(중복 금지).
    - 구현 방식은 자유(예: 증가 카운터 기반, 난수 기반 등) 단, 중복이 발생하지 않도록 보장해야 한다.

3. 역색인(Inverted Index)

- 검색 시 모든 커밋을 순회하지 않고 후보를 빠르게 가져올 수 있어야 한다.
- 키워드 추출 기준(최소 기준):
    - 커밋 메시지를 공백 기준으로 분리(split)하고, 소문자(lower)로 정규화한 토큰을 키워드로 저장한다.
- 최소 2종 인덱스를 지원한다.
    - keyword -> commit_hash 목록
    - author -> commit_hash 목록

4. 정렬 알고리즘 직접 구현

- Python 표준 정렬 API 사용 금지: `sorted()` , `list.sort()` 금지
- 비교 기준을 바꿔 정렬할 수 있어야 한다. (예: 날짜 기준, 작성자 기준)

5. 명령어 기능 요구

- INIT <user_name>
  - 저장소 초기화
  - `main` 브랜치 생성 및 HEAD 설정
  - 현재 사용자(author) 설정
- BRANCH <branch_name>
  - 현재 커밋(HEAD)을 가리키는 새 브랜치 생성
- SWITCH <branch_name>
  - HEAD를 지정한 브랜치로 이동
- COMMIT <message>
  - 현재 HEAD를 부모로 하는 새 커밋 생성
  - 생성 시 역색인(author/keyword)을 갱신
- LOG
  - 학습 목적상 “최신순 나열”이 아니라, 부모 커밋이 항상 자식 커밋보다 먼저 출력되도록 출력한다(위상 정렬 성격).
  - 출력 포맷은 자유이나, 커밋 hash / author / timestamp / message가 식별 가능해야 한다.
- LOG --sort-by=date|author
  - `date` : timestamp 기준으로 정렬(동률 처리 규칙은 자유)
  - `author` : 작성자 이름 기준으로 정렬(동률 처리 규칙은 자유)
- PATH <commit1> <commit2>
  - 최단 경로는 “커밋-부모 연결을 무방향 간선으로 간주”했을 때의 최단 경로(간선 수 최소)로 정의한다.
  - 경로가 없으면 `No path` 를 출력한다.
  - 최단 경로가 여러 개면, “경로를 `hash1->hash2->...` 문자열로 만들었을 때 사전순이 가장 작은 경로”를 선택한다.

- ANCESTORS <commit_hash>
    - 해당 커밋에서 도달 가능한 모든 조상 커밋을 빠짐없이 출력한다.
- SEARCH <keyword>
    - 키워드가 포함된 커밋 메시지의 커밋들을 출력한다(역색인 기반).
- SEARCH --author=<name>
    - 특정 작성자의 커밋들을 출력한다(역색인 기반).

5. 보너스 과제 (선택)

1. Diff(간단 비교) 추가
    - diff <file1> <file2> 명령을 추가해 두 텍스트 파일을 줄 단위로 비교한다.
    - 추가/삭제/공통 줄을 구분해 출력한다.
2. Merge(브랜치 병합) 흉내내기
    - merge <branch_name> 명령을 추가한다.
    - 부모가 2개인 merge commit을 생성한다(현재 브랜치의 HEAD와 대상 브랜치의 HEAD를 부모로).
3. 정렬 알고리즘 성능 비교
    - 서로 다른 정렬 알고리즘을 2개 이상 구현하고, 입력 크기에 따른 실행 시간을 비교한다.

개발환경

6. 개발 환경

- Python 3.10 이상

제약조건

7. 제약 사항

- **실행**
  - 실행 커맨드(예): `python main.py`
- **라이브러리 제한**
  - 그래프 전용 라이브러리 사용 금지
  - 정렬 관련 표준 API 전부 금지: `sorted()`, `list.sort()` 등
  - 기본 자료형(예: `list`, `dict`, `set`)과 문자열/파일 입출력/시간 처리는 사용 가능
- **구조/품질**
  - 알고리즘 로직(탐색/정렬/인덱싱)은 독립된 함수 또는 클래스로 분리한다.
  - 주요 함수/클래스에 주석 또는 docstring을 작성한다.
- **기능 범위**
  - 파일 내용 추적은 구현하지 않는다(커밋 메타데이터 중심).
  - 네트워크 통신은 구현하지 않는다.
  - 데이터 영속성(파일 저장)은 구현하지 않아도 된다(메모리 상 동작으로 충분).

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구와 디자인은 달라도 된

- 실행 예시(예시)

```
mini-git> init "Alice"
Initialized repository.
Current branch: main
Current user: Alice

mini-git> commit "Initial commit"
[main a1b2c3] Initial commit
```

```
mini-git> branch feature
Created branch: feature

mini-git> switch feature
Switched to branch: feature

mini-git> commit "Add login feature"
[feature d4e5f6] Add login feature

mini-git> switch main
Switched to branch: main

mini-git> commit "Add payment feature"
[main g7h8i9] Add payment feature

mini-git> log
commit a1b2c3 (Alice, 2024-01-15 09:00:00) [main]
Initial commit
commit d4e5f6 (Alice, 2024-01-15 09:15:00) [feature]
Add login feature
commit g7h8i9 (Alice, 2024-01-15 09:30:00) [main]
Add payment feature

mini-git> path a1b2c3 g7h8i9
Path: a1b2c3 -> g7h8i9

mini-git> search "login"
Found 1 commit:

- d4e5f6: Add login feature

mini-git> log --sort-by=author
commit a1b2c3 (Alice, 2024-01-15 09:00:00)
Initial commit
commit d4e5f6 (Alice, 2024-01-15 09:15:00)
Add login feature
commit g7h8i9 (Alice, 2024-01-15 09:30:00)
Add payment feature
```